

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4301

Name: R Thrisha

Reg No:192373045

COURSE OUTCOME COVERED

CO -2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Design Task

Weightage: 20%

QUESTIONS:

Total Marks: 20

Question 1: Responsive Navigation Bar Design - Marks(4)

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.

- (a) Identify key components required in a navigation bar for an e-commerce website.
 - (b) Design a responsive navigation bar using appropriate CSS techniques.
 - (c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).
 - (d) Evaluate the usability and suggest improvements for better user experience.
-

Question 2: Product Hover Effect - Marks(4)

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.

- (a) Identify suitable CSS properties for hover effects.
 - (b) Suggest appropriate **CSS animation or transition** for product interaction.
 - (c) Design a hover effect for a product card.
 - (d) Evaluate performance and user experience considerations.
-

Question 3: Layout Selection (Flexbox vs Grid) - Marks(4)

Suppose that a dashboard layout needs to be designed for a web application.

- (a) Identify layout requirements of a dashboard (rows, columns, alignment).
 - (b) Compare **Flexbox and Grid** based on layout needs.
 - (c) Choose the most suitable layout technique and justify your choice.
 - (d) Evaluate the scalability and responsiveness of the chosen approach.
-

Question 4: Mobile-First Blog Layout - Marks(4)

Suppose that a blog page must be designed following a mobile-first approach.

- (a) Identify key sections of a blog layout.
 - (b) Design a mobile-first layout using CSS.
 - (c) Demonstrate how the layout scales for larger screens using media queries.
 - (d) Evaluate accessibility and readability of the design.
-

Question 5: JavaScript Event Handling for Dynamic Menu - Marks(4)

Suppose that a website includes a dynamic menu that responds to user interactions.

- (a) Identify appropriate JavaScript events for menu interaction.
- (b) Design event handling logic for menu toggle functionality.
- (c) Implement interaction for user actions (click/hover).
- (d) Evaluate performance and suggest improvements.

Code of Conduct:

I T Gnaneswari(192372183) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

T Gnaneswari

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools (CSS/JS)	20	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10	Clear, wellstructured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Question 1: Responsive Navigation Bar Design

A responsive navigation bar is an important part of an e-commerce website. It helps users move between different pages such as Home, Products, Categories, Cart, and Login. The navigation bar should work properly on desktop, tablet, and mobile screens.

(a) Key Components Required in an E-Commerce Navigation Bar

The important components are:

1. Website Logo

Displays the company or shopping website name. Clicking the logo usually redirects the user to the home page.

2. Navigation Links

Common links include Home, Products, Categories, Offers, About Us, and Contact. 3.

Search Bar

Allows users to search for products quickly.

4. Shopping Cart Icon

Shows the number of products added to the cart and opens the cart page.

5. Login or Profile Option

Allows users to sign in, register, or access their profile and order history.

6. Hamburger Menu

A three-line menu icon displayed on smaller screens. It opens and closes the navigation links.

7. Wishlist Icon

Allows users to save products for later purchase.

(b) Responsive Navigation Bar Design

The navigation bar can be designed using HTML, CSS Flexbox, and media queries.

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Responsive E-Commerce Navbar</title>
```

```
<style>
  * {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
  }

  body {
    font-family: Arial, sans-serif;
  }

  .navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    background-color: #232f3e;
    padding: 15px 30px;
    color: white;
  }

  .logo {
    font-size: 24px;
    font-weight: bold;
  }

  .nav-links {
    display: flex;
    list-style: none;
    gap: 25px;
  }

  .nav-links a {
    color: white;
    text-
```

```
decoration: none;
font-size: 16px;
}
```

```
.nav-links a:hover {
color: #ff9900;
}
```

```
.search-box input {
padding: 8px;      border:
none;      border-radius:
4px;
}
```

```
.cart {
font-size: 18px;
}
```

```
.hamburger {
display: none;
font-size: 28px;
cursor: pointer;
}
```

```
@media screen and (max-width: 768px) {
.navbar {
flex-wrap: wrap;
}
```

```
.hamburger {
display: block;
}
```

```
        .nav-links {
display: none;
width: 100%;
flex-direction: column;
text-align: center;
margin-top: 15px;
        }
```

```
        .nav-links.active {
display: flex;
        }
```

```
        .search-box {
display: none;
        }
    }
```

```
</style>
</head>
```

```
<body>
```

```
<nav class="navbar">
    <div class="logo">ShopEasy</div>
```

```
    <div class="hamburger" id="hamburger">☰</div>
```

```
    <ul class="nav-links" id="navLinks">
<li><a href="#">Home</a></li>
        <li><a href="#">Products</a></li>
        <li><a href="#">Categories</a></li>
        <li><a href="#">Offers</a></li>
        <li><a href="#">Login</a></li>
    </ul>
```

```

<div class="search-box">
  <input type="text" placeholder="Search products"> </div>

<div class="cart"> Cart</div>
</nav>

<script>
  const hamburger = document.getElementById("hamburger");
  const navLinks = document.getElementById("navLinks");

  hamburger.addEventListener("click", function () {
    navLinks.classList.toggle("active");
  });
</script>

</body>
</html>

```

In this design, Flexbox is used to arrange the logo, navigation links, search box, and cart in a single row. The `justify-content: space-between` property creates equal spacing between the items, while `align-items: center` vertically aligns them.

(c) Navigation Adaptation for Smaller Screens

On desktop screens, all navigation links, the search box, and the cart are displayed in one row. On tablet and mobile screens, the media query is activated when the screen width becomes less than or equal to 768px.

@media screen and (max-width: 768px) The following changes occur:

- The normal navigation links are hidden.
- The hamburger icon is displayed.
- The search box is hidden to save screen space.
- The menu links are arranged vertically.

- When the user clicks the hamburger icon, JavaScript adds or removes the active class.

```
hamburger.addEventListener("click", function () {  
  navLinks.classList.toggle("active");  
});
```

When the active class is added, the menu becomes visible.

```
.nav-links.active {  
  display: flex;  
}
```

(d) Usability Evaluation and Suggested Improvements

The responsive navigation bar improves usability because users can easily access important pages on different devices. The desktop version provides complete navigation, while the mobile version saves space by using a hamburger menu.

The following improvements can provide a better user experience:

1. Use Clear Labels

Menu names such as Home, Products, Cart, and Login should be easy to understand.

2. Provide Large Touch Areas

Buttons and links should be large enough for mobile users to tap comfortably.

3. Add Accessibility Features

The hamburger button should include an aria-label so screen-reader users can understand its purpose.

```
<div class="hamburger" aria-label="Open navigation menu">☰</div>
```

4. Show Cart Item Count

A small number beside the cart icon can indicate how many products are present in the cart.

5. Add Search on Mobile

Instead of completely hiding the search bar, it can be placed inside the mobile menu.

6. Highlight the Active Page

The currently opened page should be shown using a different colour or underline.

7. Support Keyboard Navigation

Users should be able to navigate through menu items using the Tab key.

8. Use Smooth Menu Animation

A smooth opening and closing transition can improve the visual experience.

Conclusion

The responsive navigation bar uses HTML, CSS Flexbox, media queries, and JavaScript event handling. It displays a full menu on larger screens and a hamburger menu on smaller screens. This design improves navigation, saves screen space, and provides a better user experience across desktop, tablet, and mobile devices.

Question 2: Product Hover Effect

A product hover effect is a visual change that appears when the user moves the mouse pointer over a product card. It helps attract attention, improves interaction, and makes the e-commerce website look more modern and responsive.

(a) Suitable CSS Properties for Hover Effects

The following CSS properties are commonly used to create hover effects: 1.

transform

Used to move, rotate, enlarge, or reduce an element.

Example:

transform: scale(1.05); 2.

box-shadow

Adds a shadow around the product card and gives a raised appearance.

Example:

box-shadow: 0 8px 20px rgba(0, 0, 0, 0.2);

3. transition

Makes the change between the normal and hover states smooth.

Example:

transition: all 0.3s ease;

4. opacity

Changes the transparency of an image or element.

Example:

opacity: 0.8;

5. filter

Applies visual effects such as brightness, contrast, or grayscale.

Example:

filter: brightness(1.1);

6. background-color

Changes the background colour when the user hovers over the product.

7. **color**

Changes the text colour during hover.

8. **border-color**

Highlights the product card border.

These properties help provide visual feedback to users and make product items more attractive.

(b) Appropriate CSS Animation or Transition

For product interactions, CSS transitions are generally more suitable than complex animations because transitions are simple, smooth, and efficient.

A transition can be applied to the product card as follows:

```
.product-card {  
  transition: transform 0.3s ease, box-shadow 0.3s ease;  
}
```

In this example:

- transform 0.3s ease smoothly enlarges the card.
- box-shadow 0.3s ease smoothly adds a shadow.
- 0.3s is the duration of the transition.
- ease makes the animation start and end smoothly.

The hover effect can be applied using:

```
.product-card:hover {  
  transform: translateY(-8px) scale(1.03);  box-  
shadow: 0 10px 25px rgba(0, 0, 0, 0.2);  
}
```

This moves the product card slightly upward and enlarges it when the user places the mouse pointer over it.

(c) Product Card Hover Effect Design

The following example creates a product card with an image, product name, price, and Add to Cart button.

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Product Hover Effect</title>
```

```
<style>
  * {
    margin: 0;
padding: 0;      box-sizing:
border-box;
  }

  body {
    font-family: Arial, sans-serif;
background-color: #f4f4f4;
display: flex;    justify-content:
center;          align-items: center;
min-height: 100vh;
  }

  .product-card {      width: 280px;
background-color: white;    border-radius:
12px;    overflow: hidden;    text-align:
center;    padding-bottom: 20px;
box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
transition: transform 0.3s ease,
box-shadow 0.3s ease;
  }

  .product-card img {
width: 100%;    height: 220px;
object-fit: cover;    transition:
transform 0.3s ease;
  }

  .product-card h2 {
```

```
        margin: 15px 0 8px;
font-size: 22px;
    }
```

```
        .product-card p {
color: #555;        margin-
bottom: 10px;
    }
```

```
        .price {        font-
size: 20px;        font-
weight: bold;        color:
#e65100;        margin-
bottom: 15px;
    }
```

```
        .product-card button {
background-color: #ff6600;
color: white;        border: none;
padding: 10px 20px;
border-radius: 6px;        cursor:
pointer;
        transition: background-color 0.3s ease;
    }
```

```
        .product-card:hover {        transform:
translateY(-8px) scale(1.03);        box-shadow: 0
12px 25px rgba(0, 0, 0, 0.25);
    }
```

```
        .product-card:hover img {
transform: scale(1.08);
    }
```

```

        .product-card button:hover {
background-color: #cc5200;
        }
    </style>
</head>

<body>

    <div class="product-card">
        

        <h2>Wireless Headphones</h2>

        <p>High-quality sound with long battery life.</p>

        <div class="price">₹1,999</div>

        <button>Add to Cart</button>
    </div>

</body>
</html>

```

Working of the Hover Effect

In the normal state, the product card appears with a small shadow.

When the user places the mouse pointer over the product card:

- The card moves slightly upward.
- The card becomes slightly larger.
- A stronger shadow appears.
- The product image zooms in.
- The Add to Cart button changes colour when hovered.

These effects make the product card appear interactive and help attract the user's attention.

(d) Performance and User Experience Considerations

Hover effects should be designed carefully because excessive effects may reduce website performance and disturb users.

Performance Considerations 1. Use Transform and Opacity

Properties such as transform and opacity are usually smoother and more efficient than changing width, height, margin, or position. transform: scale(1.05);

2. Avoid Heavy Animations

Complex animations on many product cards may cause slow performance, especially on lowend mobile devices.

3. Use Short Transition Durations

A duration between 0.2s and 0.4s is usually suitable for hover effects. Very slow animations may frustrate users.

4. Avoid Transitioning All Properties Instead

of using:

transition: all 0.3s ease; it is better to specify

only the required properties:

transition: transform 0.3s ease, box-shadow 0.3s ease; This improves browser performance.

5. Optimize Product Images

Large image files can slow down the page. Images should be compressed and displayed in suitable dimensions.

User Experience Considerations 1. Keep Effects Simple

The hover effect should improve interaction without distracting users.

2. Maintain Readability

Text and buttons should remain clearly visible during the hover effect.

3. Provide Clear Feedback

The effect should show users that the product card is clickable or interactive.

4. Support Touch Devices

Mobile devices do not use mouse hover in the same way as desktop devices. Therefore, important actions such as Add to Cart should always remain visible.

5. Avoid Excessive Zoom

Too much scaling may overlap nearby product cards and disturb the page layout.

6. Consider Accessibility

Users who prefer reduced motion may find animations uncomfortable. The design can support them using:

```
@media (prefers-reduced-motion: reduce) {  
  .product-card,  
  .product-card img, .product-  
card button {    transition: none;  
  } }  
}
```

This removes animation for users who have enabled reduced-motion settings.

Conclusion

The product hover effect improves the appearance and interactivity of an e-commerce website. CSS properties such as transform, box-shadow, transition, opacity, and filter can be used to create smooth visual effects. The effect should be simple, fast, accessible, and suitable for both desktop and mobile users.

Question 3: Layout Selection — Flexbox vs Grid

A dashboard is a web page that displays important information such as statistics, charts, reports, notifications, and user details. Since dashboards usually contain many sections arranged in rows and columns, selecting the correct CSS layout technique is very important.

(a) Layout Requirements of a Dashboard

A dashboard generally requires the following layout features:

1. Rows and Columns

Dashboard items such as cards, charts, tables, and reports must be arranged in proper rows and columns.

2. Header Section

The top area may contain the application name, search bar, notifications, and user profile.

3. Sidebar Navigation

A vertical sidebar is commonly used for links such as Dashboard, Users, Reports, Settings, and Logout.

4. Main Content Area

The central section displays summary cards, graphs, recent activities, and tables.

5. Proper Alignment

All components should be aligned consistently so that the dashboard looks organized and professional.

6. Equal Spacing

Suitable gaps should be maintained between cards, charts, and other sections.

7. Responsive Design

The layout should adjust properly across desktop, tablet, and mobile screens.

8. Flexible Card Sizes

Some sections may require more space than others. For example, a large chart may occupy two columns, while a small statistics card may occupy one column.

(b) Comparison Between Flexbox and Grid

Both Flexbox and CSS Grid are modern CSS layout techniques, but they are used for different purposes.

Feature	Flexbox	CSS Grid
Layout Direction	One-dimensional	Two-dimensional
Arrangement	Row or column	Rows and columns
Best Use	Navigation bars, buttons, small Dashboards, galleries, complete components	page layouts
Alignment	Excellent for aligning items in one direction	Excellent for controlling rows and columns
Item Placement	Based mainly on content flow	Allows precise placement
Complexity	Simple for small layouts	Better for complex layouts
Responsiveness	Good	Very good
Overlapping Elements	Limited control	Supports overlapping and named grid areas

Flexbox

Flexbox is a one-dimensional layout system. It arranges elements either horizontally in a row or vertically in a column.

Example:

```
.container {  
  display: flex;  
  gap: 20px;
```

```
}
```

Flexbox is suitable for:

- Navigation bars
- Button groups
- Header sections
- Aligning items in one row
- Small reusable components

CSS Grid

CSS Grid is a two-dimensional layout system. It controls both rows and columns at the same time.

Example:

```
.dashboard {  
display: grid;  
  grid-template-columns: repeat(3, 1fr);  
gap: 20px;  
}
```

CSS Grid is suitable for:

- Dashboard layouts
- Product galleries
- Image galleries
- Complex web page structures
- Card-based layouts

(c) Most Suitable Layout Technique and Justification

For a dashboard layout, **CSS Grid is the most suitable technique.**

A dashboard contains multiple rows and columns. CSS Grid allows the developer to define column widths, row sizes, gaps, and component positions easily.

The following example creates a responsive dashboard:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>Dashboard Layout</title>
```

```

<style>
    * {
        margin: 0;
padding: 0;
        box-sizing:
border-box;
    }

    body {
        font-family: Arial, sans-serif;
background-color: #f4f6f8;
    }

    .dashboard {
display: grid;
        grid-template-columns: 220px 1fr;
grid-template-rows: 70px auto;
grid-template-areas:
header"
        "sidebar main";
min-height: 100vh;
    }

    .header {
        grid-area:
header;
        background-
color: white;
        padding:
20px;
        box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
    }

    .sidebar {
grid-area: sidebar;
background-color:
#263238;
        color:
white;
        padding:
20px;

```

```
}
```

```
.sidebar h2 {      margin-  
bottom: 20px;  
}
```

```
.sidebar ul {  
list-style: none;  
}
```

```
.sidebar li {  
margin: 15px 0;  
}
```

```
.main-content {  
grid-area: main;  
padding: 20px;  
}
```

```
.cards {  
display: grid;  
    grid-template-columns: repeat(3, 1fr);  
gap: 20px;  
}
```

```
.card {      background-color: white;  
padding: 25px;      border-radius: 10px;  
box-shadow: 0 3px 10px rgba(0, 0, 0, 0.1);  
}
```

```
@media screen and (max-width: 900px) {  
    .cards {  
        grid-template-columns: repeat(2, 1fr);  
    }  
}
```

```
}
```

```
@media screen and (max-width: 600px) {  
  .dashboard {  
    grid-template-columns: 1fr;  
    grid-template-rows: auto auto auto;  
    grid-template-areas:  
      "header"  
      "sidebar"  
      "main";  
  }  
}
```

```
.cards {  
  grid-template-columns: 1fr;  
}  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="dashboard">
```

```
<header class="header">
```

```
<h1>Admin Dashboard</h1>
```

```
</header>
```

```
<aside class="sidebar">
```

```
<h2>Menu</h2>
```

```
<ul>
```

```
<li>Dashboard</li>
```

```
<li>Users</li>
```

```
<li>Reports</li>
```

```
<li>Settings</li>
```

```
</ul>
</aside>

<main class="main-content">
  <div class="cards">
    <div class="card">
      <h3>Total Users</h3>
      <p>1,250</p>
    </div>

    <div class="card">
      <h3>Total Sales</h3>
      <p>₹85,000</p>
    </div>

    <div class="card">
      <h3>Orders</h3>
      <p>320</p>
    </div>
  </div>
</main>
```

```
</div>
```

```
</body>
```

```
</html>
```

Justification

CSS Grid is selected because:

- It manages both rows and columns.
- It allows precise control over dashboard sections.
- It supports named areas such as header, sidebar, and main.
- It makes the code easier to understand and maintain.
- It supports responsive layouts using media queries.
- It can automatically adjust the number of columns.

Flexbox can still be used inside individual dashboard components, such as aligning buttons, icons, or navigation items. Therefore, a practical design may use Grid for the main layout and Flexbox for smaller internal components.

(d) Scalability and Responsiveness of CSS Grid

CSS Grid provides good scalability because new cards, charts, and widgets can be added without redesigning the entire layout.

For example:

```
.cards {  
display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));  
gap: 20px;  
}
```

In this code:

- auto-fit automatically calculates how many columns can fit.
- minmax(250px, 1fr) ensures that each card has a minimum width of 250 pixels.
- 1fr allows cards to expand equally.
- gap provides consistent spacing.

Responsiveness

On desktop screens, the dashboard may display three or four columns.

On tablet screens, it can display two columns.

On mobile screens, the cards can be arranged in a single column.

This can be achieved using media queries:

```
@media screen and (max-width: 900px) {  
  .cards {  
    grid-template-columns: repeat(2, 1fr);  
  }  
}
```

```
@media screen and (max-width: 600px) {  
  .cards {  
    grid-template-columns: 1fr;  
  }  
}
```

Scalability Benefits 1. Easy Addition of Components

New widgets can be added without changing the complete structure.

2. **Automatic Space Management**

Grid distributes available space properly.

3. **Consistent Alignment**

Cards remain aligned even when their content differs.

4. **Better Code Maintenance**

The layout code remains clean and understandable.

5. **Supports Complex Designs**

Large charts, tables, and cards can occupy multiple columns.

Example:

```
.large-chart { grid-  
column: span 2;  
}
```

6. **Device Compatibility**

Modern browsers support CSS Grid, making it suitable for current web applications.

Conclusion

Flexbox is best suited for one-dimensional layouts such as navigation bars and small component alignment. CSS Grid is more suitable for dashboard layouts because it manages rows and columns simultaneously. It provides better control, responsiveness, scalability, and maintainability. Therefore, CSS Grid is the preferred layout technique for designing a modern web application dashboard.

Question 4: Mobile-First Blog Layout

A mobile-first approach means designing the blog page for small mobile screens first and then improving the layout for tablets and desktop screens. This approach provides better performance, readability, and usability for mobile users.

(a) Key Sections of a Blog Layout

A blog page generally contains the following sections: 1.

Header

Displays the blog name, logo, and website title.

2. Navigation Bar

Contains links such as Home, Articles, Categories, About, and Contact.

3. Main Article Section

Displays the blog title, author name, publication date, article image, and article content.

4. Sidebar

Contains recent posts, categories, popular articles, search options, or advertisements.

5. Related Posts Section

Suggests other articles related to the current blog post.

6. Comment Section

Allows readers to share opinions and interact with the author.

7. Footer

Contains copyright information, contact details, social media links, and privacy-policy links.

For mobile screens, these sections are normally displayed in a single-column layout.

(b) Mobile-First Layout Using CSS

In a mobile-first design, the default CSS is written for small screens. Additional media queries are then used for larger screens.

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport"
```

```
  content="width=device-width, initial-scale=1.0">
```

```
<title>Mobile-First Blog</title>
```

```
<style>
```

```
  * {      margin: 0;
padding: 0;      box-sizing:
border-box;
  }
```

```
body {      font-family:
Arial, sans-serif;      line-height:
1.6;      background-color:
#f4f4f4;      color: #222;
}
```

```
header {      background-
color: #263238;      color:
white;      text-align: center;
padding: 20px;
}
```

```
nav {      background-
color: #37474f;      padding:
10px;
}
```

```
nav ul {      list-
style: none;      text-
align: center;
}
```

```
nav li {
margin: 10px 0;
}
```

```
nav a {      color:
white;      text-
decoration: none;
}
```

```
.container {
width: 100%;
padding: 15px;
```

```
}
```

```
    main,    aside {  
background-color: white;  
padding: 20px;    margin-  
bottom: 20px;    border-  
radius: 8px;  
    }
```

```
    article img {  
width: 100%;  
height: auto;  
margin: 15px 0;  
border-radius: 6px;  
    }
```

```
    article h2 {    margin-  
bottom: 10px;  
    }
```

```
    article p  
{  
margin-  
bottom: 15px;  
    }
```

```
    aside ul {  
padding-left: 20px;  
    }
```

```
    footer {  
        background-color: #263238;  
color: white;    text-align:  
center;    padding: 15px;
```

```
    }
</style>
</head>

<body>

<header>
  <h1>My Technology Blog</h1>
  <p>Learn web development and programming</p>
</header>

<nav>
  <ul>
    <li><a href="#">Home</a></li>
    <li><a href="#">Articles</a></li>
    <li><a href="#">Categories</a></li>
    <li><a href="#">About</a></li>
  </ul>
</nav>

<div class="container">
  <main>
    <article>
      <h2>Introduction to Responsive Web Design</h2>

      <p>
        Written by Ganeswari |
        Published on August 3, 2026
      </p>

      

      <p>
```

Responsive web design helps web pages adjust properly across mobile, tablet, and desktop screens.

Mobile-first design begins with a simple single-column layout and adds advanced styles for larger screens.

```
</p>

<p>
  Mobile-first design begins with a simple
  single-column layout and adds advanced styles      for
  larger screens.
</p>
</article>
</main>

<aside>
  <h3>Recent Posts</h3>

  <ul>
    <li>Introduction to HTML</li>
    <li>CSS Flexbox Guide</li>
    <li>JavaScript Events</li>
  </ul>
</aside>
</div>
<footer>
  <p>&copy; 2026 My Technology Blog</p>
</footer>
</body>
</html>
```

Explanation of the Mobile Layout

The layout is designed for mobile screens by default.

- The navigation links appear vertically.
- The main article and sidebar appear one below the other.
- The image uses width: 100%, so it fits the screen.
- The page uses sufficient padding around the content.
- The font has a line height of 1.6 for better readability.

- The layout does not create horizontal scrolling.

This creates a simple single-column layout suitable for mobile devices.

(c) Scaling the Layout for Larger Screens

Media queries are used to change the layout when the screen becomes wider.

The following CSS can be added below the mobile-first CSS:

```
/* Tablet layout */
```

```
@media screen and (min-width: 600px) {
```

```
    nav ul {      display:
flex;      justify-content:
center;      gap: 30px;
```

```
    }    nav li {
margin: 0;
    }
```

```
    .container {
padding: 25px;
    }
}
```

```
/* Desktop layout */
```

```
@media screen and (min-width: 900px) {
```

```
    .container {
display: grid;
    grid-template-columns: 3fr 1fr;
gap: 25px;    max-width:
1200px;    margin: auto;
    }
```

```
    main,    aside {
margin-bottom: 0;
    }
```

```
header {  
padding: 35px;  
}  
}
```

Tablet Screens

When the screen width is at least 600px:

- The navigation links are displayed horizontally.
- More spacing is added between navigation items.
- The content receives additional padding.

Desktop Screens

When the screen width is at least 900px:

- CSS Grid divides the page into two columns.
- The main article occupies three portions of the available space.
- The sidebar occupies one portion.
- The layout is limited to a maximum width of 1200px.
- The content is centred using margin: auto.

The property below creates the desktop columns: grid-template-columns: 3fr 1fr;

This means the article section is three times wider than the sidebar.

Therefore, the layout changes as follows:

- **Mobile:** One column
- **Tablet:** Wider single column with horizontal navigation
- **Desktop:** Two columns with article and sidebar

(d) Accessibility and Readability Evaluation

Accessibility ensures that the blog can be used by people with different abilities. Readability ensures that users can comfortably understand the content.

1. Responsive Viewport

The viewport meta tag ensures that the page fits correctly on mobile devices.

```
<meta name="viewport"  
content="width=device-width, initial-scale=1.0">
```

Without this tag, mobile browsers may display the page at a desktop width.

2. Readable Font Size

The body text should normally use a minimum font size of approximately 16px.

```
body { font-size: 16px;
```

```
}
```

Small text can be difficult to read, particularly on mobile devices.

3. Proper Line Spacing

The following property improves paragraph readability:

```
line-height: 1.6;
```

It prevents lines of text from appearing too close together.

4. Sufficient Colour Contrast

Text and background colours should have good contrast. For example, dark text on a white background is easier to read.

Light-grey text on a white background should be avoided because it may be difficult for users with visual impairments.

5. Meaningful Alternative Text

Every important image should contain meaningful alt text.

```

```

Screen readers use alternative text to explain images to visually impaired users.

6. Semantic HTML Elements

Semantic elements such as the following improve structure:

```
<header>
<nav>
<main>
<article>
<aside>
<footer>
```

These elements help browsers, search engines, and assistive technologies understand the page.

7. Keyboard Accessibility

All navigation links should be accessible using the Tab key. A visible focus style should be provided: `a:focus {`

```
    outline: 3px solid #ff9800;    outline-
offset: 3px;
}
```

This helps keyboard users identify the currently focused link.

8. Comfortable Line Length

On large screens, paragraphs should not stretch across the entire screen. Very long lines are difficult to read.

```
article p { max-  
width: 70ch;  
}
```

The ch unit helps maintain a comfortable number of characters per line.

9. Clear Heading Structure

Headings should be used in the correct order:

- <h1> for the blog website title
- <h2> for the article title
- <h3> for sidebar or subsection headings

Correct heading order helps users and screen readers understand the content structure.

10. Adequate Touch Targets

Mobile navigation links and buttons should have enough padding so that users can tap them easily.

```
nav a {  
  display: inline-block;  
  padding: 10px 15px;  
}
```

11. Avoid Horizontal Scrolling

Images and containers should not exceed the screen width. img

```
{  
  max-width: 100%;  
  height: auto;  
}
```

12. Use Simple and Clear Content

Short paragraphs, descriptive headings, lists, and sufficient spacing improve the readability of blog content.

Performance Considerations

A mobile-first layout can also improve page performance because it initially loads a simple design for small devices.

To improve performance further:

- Compress blog images.
- Use modern image formats such as WebP.
- Avoid unnecessary CSS libraries.
- Minify CSS and JavaScript files.
- Use lazy loading for images.

Example:

```

```

Conclusion

The mobile-first blog layout begins with a simple single-column structure for mobile users. CSS media queries are used to create horizontal navigation on tablets and a two-column article-and-sidebar layout on desktop screens. The design uses responsive images, readable typography, semantic HTML, accessible navigation, proper colour contrast, and sufficient spacing. Therefore, the blog remains usable, readable, and responsive across different devices.

Question 5: JavaScript Event Handling for Dynamic Menu

A dynamic menu responds to user actions such as clicking, hovering, pressing keyboard keys, or selecting menu items. JavaScript event handling is used to detect these actions and change the menu's behaviour.

For example, a dynamic menu can:

- Open and close when the user clicks a hamburger icon.
- Display a submenu when the user hovers over a menu item.
- Close when the user clicks outside the menu.
- Support keyboard navigation.
- Update accessibility attributes when the menu state changes.

(a) Appropriate JavaScript Events for Menu Interaction

The following JavaScript events are suitable for dynamic menu interaction:

1. click

The click event is used when the user clicks a menu button or hamburger icon.

```
menuButton.addEventListener("click", function () {
    menu.classList.toggle("active");
});
```

This event is suitable for:

- Opening and closing the menu
- Selecting menu items
- Opening submenus
- Closing pop-up menus

2. mouseover

The mouseover event occurs when the mouse pointer enters an element or one of its child elements.

```
menuItem.addEventListener("mouseover", function () {  
    submenu.classList.add("show");  
});
```

It can be used to display a submenu when the user hovers over a menu item.

3. mouseout

The mouseout event occurs when the mouse pointer leaves an element.

```
menuItem.addEventListener("mouseout", function () {  
    submenu.classList.remove("show");  
});
```

It can be used to hide the submenu.

4. mouseenter

The mouseenter event is similar to mouseover, but it does not repeatedly trigger when the pointer moves between child elements.

```
menuItem.addEventListener("mouseenter", function () {  
    submenu.classList.add("show");  
});
```

For menu interactions, mouseenter is often better than mouseover.

5. mouseleave

The mouseleave event is used when the pointer leaves the menu item.

```
menuItem.addEventListener("mouseleave", function () {  
    submenu.classList.remove("show");  
});
```

6. keydown

The keydown event supports keyboard interaction.

```
menuButton.addEventListener("keydown", function (event) {  
  if (event.key === "Enter") {  
    menu.classList.toggle("active");  
  }  
});
```

Useful keyboard keys include:

- Enter to open or select the menu
- Space to toggle the menu
- Escape to close the menu
- Arrow keys to move between menu items

7. focus and blur

These events are useful for keyboard users.

- focus occurs when an element receives keyboard focus.
- blur occurs when it loses focus.

They can be used to show or hide submenus for users who navigate with the Tab key.

(b) Event Handling Logic for Menu Toggle Functionality

The menu toggle logic follows these steps:

1. Select the hamburger button.
2. Select the navigation menu.
3. Listen for the button click.
4. Add or remove an active class.
5. Update the menu's accessibility state.

Example: const menuButton =

```
document.getElementById("menuButton"); const navigationMenu =
```

```
document.getElementById("navigationMenu");
```

```
menuButton.addEventListener("click", function () {  
  navigationMenu.classList.toggle("active");
```

```
    const isOpen = navigationMenu.classList.contains("active");
```

```
    menuButton.setAttribute("aria-expanded", isOpen);
});
```

The `classList.toggle()` method adds the active class when it is absent and removes it when it is already present.

The following CSS controls the menu visibility:

```
.navigation-menu {
display: none;
}
```

```
.navigation-menu.active {
display: block; }
```

The `aria-expanded` attribute tells screen-reader users whether the menu is open or closed.

Initially:

```
aria-expanded="false" When
the menu opens: aria-
expanded="true"
```

(c) Dynamic Menu with Click and Hover Interaction

The following complete program demonstrates:

- Hamburger menu toggle using click
- Submenu display using hover
- Keyboard support
- Closing the menu using the Escape key

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport"
```

```
    content="width=device-width, initial-scale=1.0">
```

```
  <title>Dynamic Menu</title>
```

```
  <style>
```

```
    * {          margin: 0;
padding: 0;          box-sizing:
border-box;
    }
```

```
    body {
        font-family: Arial, sans-serif;
    }
```

```
    .navbar {
display: flex;
        justify-content: space-between;
align-items: center;          background-
color: #263238;          padding: 15px 25px;
    }
```

```
    .logo {
color: white;
font-size: 24px;
font-weight: bold;
    }
```

```
    .menu-button {
display: none;
background: none;
border: none;          color:
white;          font-size:
28px;          cursor:
pointer;
    }
```

```
    .navigation-menu {
display: flex;          list-
```

```
style: none;      gap:
25px;
}
```

```
.navigation-menu li {
position: relative;
}
```

```
.navigation-menu a {
color: white;      text-
decoration: none;
padding: 10px;     display:
block;
}
```

```
.navigation-menu a:hover, .navigation-
menu a:focus {      background-color:
#455a64;
}
```

```
.submenu {
display: none;
position: absolute;
top: 100%;         left: 0;
min-width: 160px;
list-style: none;
background-color: #37474f;
}
```

```
.submenu.show {
display: block;
}
```

```
@media screen and (max-width: 768px) {
```

```

        .menu-button {
display: block;
        }

        .navigation-menu {
display: none;          position:
absolute;              top: 60px;
left: 0;               width: 100%;
flex-direction: column;      gap:
0;                    background-color:
#263238;
        }

        .navigation-menu.active {
display: flex;
        }

        .navigation-menu li {
width: 100%;           text-align:
center;
        }

        .submenu {
position: static;
width: 100%;
        }
    }
</style>
</head>

<body>

<nav class="navbar">
    <div class="logo">ShopZone</div>

```



```
<button      class="menu-button"
id="menuButton"      aria-label="Open
navigation menu"      aria-
expanded="false"      aria-
controls="navigationMenu">
```

≡

```
</button>
```

```
<ul class="navigation-menu" id="navigationMenu">      <li>
    <a href="#">Home</a>
</li>
```

```
<li class="dropdown" id="productMenu">
    <a href="#" aria-haspopup="true">
        Products
    </a>
```

```
<ul class="submenu" id="productSubmenu">
    <li><a href="#">Mobiles</a></li>
    <li><a href="#">Laptops</a></li>
    <li><a href="#">Accessories</a></li>
</ul>
</li>
```

```
<li>
    <a href="#">Offers</a>
</li>
```

```
<li>
    <a href="#">Contact</a>
</li>
</ul>
```

```
</nav>
```

```
<script>  const
```

```
menuButton =
```

```
    document.getElementById("menuButton");
```

```
const navigationMenu =
```

```
    document.getElementById("navigationMenu");
```

```
const productMenu =    document.getElementById("productMenu");
```

```
const productSubmenu =
```

```
    document.getElementById("productSubmenu");
```

```
    menuButton.addEventListener("click", function () {
```

```
navigationMenu.classList.toggle("active");
```

```
    const isMenuOpen =
```

```
        navigationMenu.classList.contains("active");
```

```
        menuButton.setAttribute(
```

```
"aria-expanded",    isMenuOpen
```

```
    );
```

```
    });
```

```
    productMenu.addEventListener(
```

```
        "mouseenter",
```

```
function () {
```

```
    productSubmenu.classList.add("show");
```

```
    }
```

```
);
```

```
    productMenu.addEventListener(
```

```
"mouseleave",    function () {
```

```
        productSubMenu.classList.remove("show");
    }
);
```

```
productMenu.addEventListener(
    "focusin",
function () {
productSubMenu.class
List.add("show");

    }
);
```

```
productMenu.addEventListener(
    "focusout",
function () {
    productSubMenu.classList.remove("show");
    }
);
```

```
document.addEventListener("keydown",
function (event) {
    if (event.key === "Escape")
    {
navigationMenu.classList.remove("active");
productSubMenu.classList.remove("show");
    }
});
```

```
menuButton.setAttribute(
    "aria-expanded",
    "false"
);
```

```
menuButton.focus();
    }
}
```

```

    );

    document.addEventListener(
        "click",    function (event) {        const
clickedInsideMenu =
navigationMenu.contains(event.target);
const clickedMenuButton =
menuButton.contains(event.target);
        if
(
            !clickedInsideMenu &&
            !clickedMenuButton
        ) {
            navigationMenu.classList.remove("active");

            menuButton.setAttribute(
                "aria-expanded",
                "false"
            );
        }
    }
);
</script>

```

```

</body>

```

```

</html>

```

Working of the Dynamic Menu Menu Toggle

When the user clicks the hamburger button, the active class is added or removed.

```

navigationMenu.classList.toggle("active");

```

When the class is added, the menu becomes visible.

```

.navigation-menu.active {
display: flex;
}

```

Hover Interaction

When the pointer enters the Products menu, the submenu is displayed.

```
productMenu.addEventListener("mouseenter", function () {  
  productSubMenu.classList.add("show");  
});
```

When the pointer leaves the Products menu, the submenu is hidden.

```
productMenu.addEventListener("mouseleave", function () {  
  productSubMenu.classList.remove("show");  
});
```

Keyboard Interaction

When the user presses the Escape key, both the main menu and submenu are closed. if

```
(event.key === "Escape") {  
  navigationMenu.classList.remove("active");  
}
```

Outside Click Interaction

The menu closes when the user clicks outside the navigation area. This improves usability because users do not need to click the hamburger button again.

(d) Performance Evaluation and Suggested Improvements

Dynamic menus are generally lightweight, but poorly written event handling can reduce performance.

1. Avoid Too Many Event Listeners

Attaching separate event listeners to every menu item may increase memory usage when the menu contains many items.

Instead, event delegation can be used by attaching one event listener to the parent menu.

```
navigationMenu.addEventListener("click", function (event) {  const selectedItem =  
  event.target.closest("a");  
  
  if (selectedItem) {  
    console.log("Menu item selected");  
  } });
```

This approach is more efficient because only one event listener is used.

2. Use mouseenter Instead of mouseover

The mouseover event may run repeatedly when the pointer moves between child elements. mouseenter is more suitable because it triggers only when the pointer enters the main element.

3. Avoid Unnecessary DOM Manipulation

Repeatedly adding, removing, or creating many HTML elements can slow down the page.

Using CSS classes is more efficient:

```
menu.classList.toggle("active");
```

This is better than directly changing many style properties with JavaScript.

4. Use CSS for Visual Effects

JavaScript should control the menu state, while CSS should control its appearance and animation. .navigation-menu {

```
    transition: max-height 0.3s ease;
```

```
}
```

This keeps the code simple and improves maintainability.

5. Support Keyboard Users The

menu should work with:

- Tab key
- Enter key
- Space key
- Escape key
- Arrow keys when required

This improves accessibility for users who cannot use a mouse.

6. Use Proper ARIA Attributes

Useful accessibility attributes include:

```
aria-expanded="false" aria-
```

```
controls="navigationMenu" aria-haspopup="true"
```

These help screen readers understand the menu structure and current state.

7. Provide Large Touch Targets

Menu links should contain enough padding for mobile users.

```
.navigation-menu a {
```

```
padding: 12px 16px;
```

```
}
```

8. Avoid Hover-Only Functionality

Mobile devices do not support hover in the same way as desktop computers. Important submenus should also open through click or keyboard input.

9. Close the Menu After Selection

On mobile devices, the menu can close automatically after a user selects a link.

```
navigationMenu.addEventListener("click", function (event) {    if
(event.target.matches("a")) {
        navigationMenu.classList.remove("active");    menuButton.setAttribute("aria-
expanded", "false");
    }
});
```

10. Avoid Excessive Animations

Long or complex animations can make the menu feel slow. A short duration between 0.2s and 0.4s is normally sufficient.

Conclusion

JavaScript event handling makes a navigation menu interactive by responding to clicks, hover actions, keyboard input, and outside clicks. Events such as click, mouseenter, mouseleave, keydown, focusin, and focusout can be used to create a dynamic menu. Efficient use of event delegation, CSS classes, accessibility attributes, and keyboard support improves performance, usability, and maintainability.